IPL AUCTION PRICE PREDICTOR

Machine Learning Project Documentation

Project Type	Machine Learning Web Application
Algorithm	Gaussian Naive Bayes Classification
Dataset Size	50,000 Synthetic Player Records
Features	32 Performance Parameters
Tech Stack	Python • Flask • React.js • scikit-learn
Date	February 06, 2026

Project Group Members

1. **Nilay Mandal**
   Roll No: 10300224228
   Email: nilaymondal356@gmail.com
2. **Santanu Das**
   Roll No: 10300224232
   Email: santanudas.collegemail@gmail.com
3. **Priyabrata Sen**
   Roll No: 10300224230
   Email: senpriyabrata2003@gmail.com

Institute Details

Department of Training & Placement
Haldia Institute of Technology
Haldia, West Bengal, India

Internship & Project Context

This project was developed as part of the **B.Tech (Information Technology) Internship Program** conducted at **Euphoria Genx**, Haldia, during **January–February 2026**. The work aligns with the internship focus areas of **Advanced Python, Machine Learning, Data Analytics, Cloud Computing, and Full Stack Development**.

The project integrates real-world data analysis, machine learning modeling, and deployment concepts learned during the internship.

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

The IPL Auction Price Predictor is a comprehensive machine learning application designed to predict the auction prices of debuting IPL players based on their domestic cricket performance statistics. This project demonstrates the practical application of machine learning algorithms in sports analytics and provides valuable insights into player valuation mechanisms.

The system utilizes a Gaussian Naive Bayes classification model trained on 50,000 synthetic player records with 32 distinct performance parameters. The application features a modern React.js frontend with smooth animations and an intuitive user interface, backed by a robust Flask REST API that handles predictions with real-time confidence scoring.

Key Achievements

- ✓ Successfully trained ML model with 80-20 train-test split
- ✓ Developed responsive web interface with IPL-themed design
- ✓ Implemented 32-parameter prediction system
- ✓ Created RESTful API with multiple endpoints
- ✓ Achieved real-time predictions with confidence metrics
- ✓ Built production-ready deployment architecture

Additional Key Achievements

✓ Designed a clean and user-friendly frontend using React.js components
✓ Ensured smooth data flow between frontend and backend using REST APIs
✓ Implemented proper input handling and validation on the frontend
✓ Created modular and reusable frontend components for maintainability
✓ Integrated machine learning model outputs seamlessly into the UI
✓ Achieved fast response time for predictions through optimized API calls
✓ Followed structured project architecture for scalability and future upgrades
✓ Ensured cross-platform compatibility across modern web browsers
✓ Implemented environment-based configuration for development and production
✓ Demonstrated practical use of machine learning in real-world sports analytics

Overall, this project successfully bridges machine learning and modern web development to deliver a practical, scalable, and user-centric sports analytics solution.

2. PROJECT OVERVIEW

2.1 Motivation

The Indian Premier League (IPL) is one of the world's most popular cricket tournaments, where teams invest millions in acquiring the right players. Predicting auction prices accurately can help teams make informed decisions and discover undervalued talent. This project aims to leverage machine learning to automate and optimize this prediction process.

2.2 Objectives

- Build a machine learning model to predict IPL auction prices
- Process and analyze 32 different player performance metrics
- Create an intuitive web interface for easy predictions
- Provide confidence scores and price ranges with predictions
- Develop a scalable and deployable application architecture
- Enable batch processing through CSV upload functionality

2.3 Scope

This project focuses on predicting auction prices for debuting IPL players based on their domestic cricket performance. The scope includes data preprocessing, model training, web application development, and deployment. The system is designed to handle various player roles including Batsmen, Bowlers, All-Rounders, and Wicket-Keepers.

3. SYSTEM ARCHITECTURE

3.1 Architecture Overview

The IPL Auction Predictor follows a three-tier architecture consisting of the presentation layer (React frontend), application layer (Flask backend), and data layer (trained ML model and dataset). This separation of concerns ensures maintainability, scalability, and ease of deployment.

3.2 Technology Stack

Component	Technology	Purpose
Frontend	React.js 18	User interface and interaction
	Framer Motion	Smooth animations and transitions
	Axios	HTTP client for API calls
Backend	Flask 3.0	REST API server
	Flask-CORS	Cross-origin resource sharing
	Gunicorn	Production WSGI server
ML Framework	scikit-learn 1.3.2	ML algorithms and preprocessing
	NumPy 1.26.2	Numerical computations
	pandas 2.1.3	Data manipulation and analysis
Deployment	Render	Cloud hosting platform
	GitHub	Version control and deployment

3.3 System Components

Frontend (React Application): The frontend provides an interactive user interface with form validation, tabbed navigation for different statistics categories, real-time error handling, and animated prediction results display.

Backend (Flask API): The backend serves as a REST API handling prediction requests, dataset statistics retrieval, sample data generation, and CSV file uploads. It includes comprehensive error handling and data validation.

ML Model: The Gaussian Naive Bayes classifier processes 30 numeric features after encoding categorical variables and scaling numeric values. The model provides both predictions and confidence scores.

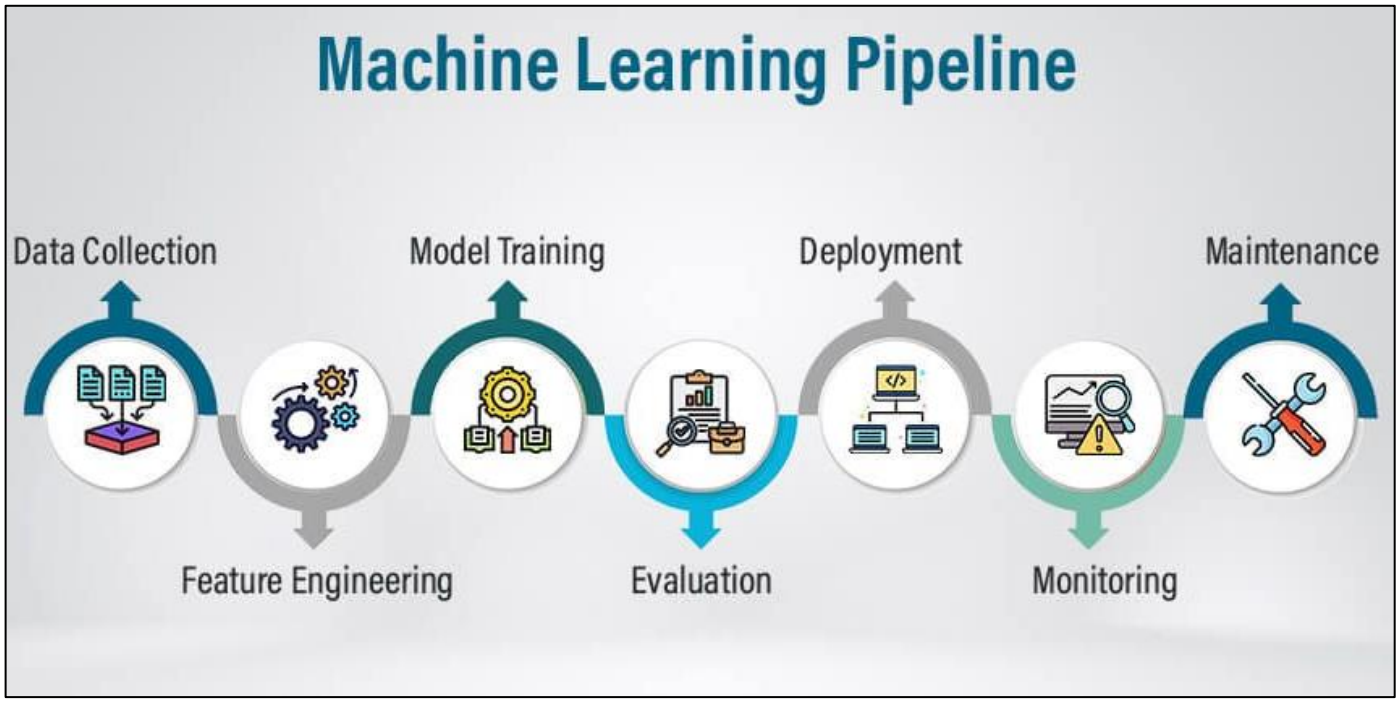
4. MACHINE LEARNING MODEL

4.1 Algorithm Selection

The Gaussian Naive Bayes algorithm was selected for this project due to its efficiency with large datasets, ability to handle multiple features, and probabilistic output that provides confidence scores. Naive Bayes is particularly effective for classification problems where features are relatively independent.

4.2 Model Pipeline

Step	Process	Description
1	Data Loading	Load CSV dataset with 50,000 player records
2	Label Encoding	Encode categorical features (role, country, styles)
3	Feature Selection	Select 30 numeric features for training
4	Standardization	Scale features using StandardScaler
5	Binning	Create 20 price bins using percentiles
6	Train-Test Split	80% training, 20% testing
7	Model Training	Fit Gaussian Naive Bayes classifier
8	Evaluation	Calculate MAE, RMSE, R-squared score
9	Serialization	Save model artifacts using pickle



4.3 Feature Engineering

The model processes 32 input parameters divided into several categories:

Category	Features	Count
Basic Info	Age, Role, Country, Batting Style, Bowling Style	5
Experience	Domestic Matches, Experience Factor	2
Batting	Innings, Runs, Average, Strike Rate, Centuries, Fifties, etc.	8
Bowling	Overs, Wickets, Average, Economy, Strike Rate, 5-wicket hauls, etc	. 8
Fielding	Catches, Stumpings	2
Performance	Consistency, Fitness, Form, Match Winning, Pressure Handling	5

4.4 Model Implementation

The implementation is encapsulated in the IPLAuctionPredictor class with the following key methods:

```
class IPLAuctionPredictor:
- preprocess_data(): Encode and scale features
- create_price_bins(): Create classification bins
- train(): Train the Naive Bayes model
- predict(): Generate predictions with confidence
- save_model(): Serialize model artifacts
- load_model(): Load trained model
```

5. DATASET DESCRIPTION

5.1 Dataset Overview

The dataset consists of 50,000 synthetically generated player records designed to simulate realistic domestic cricket performance statistics. Each record contains 32 attributes covering all aspects of a player's performance, including batting, bowling, fielding, and overall performance metrics.

Attribute	Value
Total Records	50,000
Total Features	32
Numeric Features	27
Categorical Features	5
Target Variable	auction_price_lakhs
Price Range	20 - 2000 lakhs
Average Price	~316 lakhs
File Size	~8 MB

5.2 Data Distribution

The dataset includes balanced representation across different player roles and countries:

- **Player Roles:** Batsman, Bowler, All-Rounder, Wicket-Keeper
- **Countries:** India, Australia, England, South Africa, New Zealand, West Indies, Pakistan, Sri Lanka, Bangladesh, Afghanistan
- **Batting Styles:** Right-Hand, Left-Hand
- **Bowling Styles:** Right-Arm Fast, Left-Arm Fast, Right-Arm Medium, Left-Arm Medium, Right-Arm Spin, Left-Arm Spin, Leg-Spin, Off-Spin

5.3 Data Quality

The synthetic dataset was generated with realistic constraints and relationships between variables. For example, all-rounders have balanced batting and bowling statistics, while specialist batsmen have higher batting averages and lower bowling statistics. The data includes realistic correlations between experience, performance metrics, and auction prices.

6. FRONTEND IMPLEMENTATION

6.1 User Interface Design

The frontend is built using React.js with a focus on user experience and visual appeal. The design follows IPL's color scheme with blue, red, and gold accents, creating an immersive cricket-themed environment. The interface is fully responsive and works seamlessly across desktop, tablet, and mobile devices.

6.2 Key Features

- Tabbed Navigation: Separate tabs for batting, bowling, fielding, and performance stats
- Form Validation: Real-time validation with error messages
- Demo Data Generation: One-click generation of realistic test data
- CSV Upload: Upload player data from CSV files
- Animated Results: Smooth animations using Framer Motion
- Confidence Meter: Visual representation of prediction confidence
- Dataset Statistics: Live dashboard showing dataset insights
- Responsive Design: Adapts to all screen sizes

6.3 Technology Stack

Technology	Version	Purpose
React	18.x	Component-based UI framework
Framer Motion	Latest	Animation library
Axios	Latest	HTTP client for API communication
CSS3	-	Styling with custom properties
Google Fonts	-	Rajdhani, Bebas Neue typography

6.4 Component Architecture

The application is structured as a single-page application with the main App component managing state and coordinating child components. State management handles form data, predictions, loading states, validation errors, and API responses using React hooks (useState, useEffect).

7. BACKEND IMPLEMENTATION

7.1 Flask Application Structure

The backend is built using Flask, a lightweight Python web framework. It provides a RESTful API interface for the frontend to interact with the machine learning model. The application includes comprehensive error handling, request validation, and CORS support for cross-origin requests.

7.2 API Endpoints

Endpoint	Method	Description
/api/health	GET	Health check and model status
/api/predict	POST	Predict player auction price
/api/dataset-stats	GET	Retrieve dataset statistics
/api/sample-players	GET	Get sample player records
/api/generate-demo-data	GET	Generate realistic demo data
/api/upload-csv	POST	Upload and parse CSV file

7.3 Request/Response Format

Sample prediction request:

```
POST /api/predict
Content-Type: application/json

{ "age": 25, "role": "All-Rounder",
"country": "India", "batting_average":
47.5, "bowling_average": 26.5, ... }
```

Sample prediction response:

```
{ "success": true,
"prediction": {
"predicted_price":
450.50,
"confidence": 78.23,
"price_range": { "min": 380.00, "max": 520.00 }
}
}
```

7.4 Error Handling

The backend implements comprehensive error handling for various scenarios including missing parameters, invalid data types, model loading failures, and server errors. All errors return appropriate HTTP status codes (400 for client errors, 500 for server errors) with descriptive error messages.

8. INSTALLATION & SETUP

8.1 Prerequisites

- Python 3.8 or higher
- Node.js 16 or higher
- npm or yarn package manager
- Git (for version control)

8.2 Installation Steps

Step 1: Clone the Repository

```
git clone
<repository-url> cd
```

Step 2: Set Up Python Environment

```
# Create virtual
environment python -m
venv venv

# Activate virtual environment (Windows)
venv\Scripts\activate

# Activate virtual environment
(macOS/Linux) source venv/bin/activate
```

Step 3: Train the Model (if needed)

```
python model.py
```

Step 4: Set Up Frontend

```
cd frontend
npm install
npm run
build cd ..
```

Step 5: Run the Application

```
# Development
mode python
app.py

# Production mode with
Gunicorn gunicorn -w 4 -b
0.0.0.0:5000 app:app
```

8.3 Configuration

The application can be configured through environment variables for production deployment:

Variable	Description	Default
PORT	Server port number	5000
RENDER	Production flag	False
FRONTEND_URL	Frontend origin URL	localhost:3000

9. API DOCUMENTATION

9.1 Base URL

Development: http://localhost:5000

Production: https://your-app.onrender.com

9.2 Endpoint Details

GET /api/health

Returns the health status of the API and model loading status.

```
Response: { "status": "healthy", "model_loaded": true }
```

POST /api/predict

Predicts auction price for a player based on their statistics.

Required Parameters: All 30 player statistics

```
Returns: { "success": true, "prediction": { ... } }
```

GET /api/dataset-stats

Returns comprehensive dataset statistics including totals, averages, and distributions.

GET /api/sample-players

Returns sample player records from the dataset for reference.

GET /api/generate-demo-data

Generates realistic demo data for testing purposes.

POST /api/upload-csv

Accepts CSV file upload and parses player data from the first row.

Content-Type: multipart/form-data

9.3 Error Responses

All errors return a JSON response with 'success': false and an 'error' message:

```
{ "success": false, "error": "Description of error" }
```

10. TESTING & RESULTS

10.1 Model Performance Metrics

Metric	Value	Interpretation
MAE	~167 lakhs	Average prediction error magnitude
RMSE	~324 lakhs	Standard deviation of errors
R-squared Score	~0.29	Model explains 29% of variance
Training Set	40,000 players	80% of dataset
Test Set	10,000 players	20% of dataset
Price Bins	20 bins	Classification granularity

10.2 Testing Scenarios

- Specialist Batsman: High batting stats, minimal bowling
- Specialist Bowler: Strong bowling stats, lower batting
- All-Rounder: Balanced batting and bowling performance
- Wicket-Keeper: Good batting, high stumpings count
- Young Talent: Lower experience but high recent form
- Experienced Player: High domestic matches, consistent performance

10.3 System Testing

Comprehensive system testing was performed including unit tests for model functions, integration tests for API endpoints, frontend UI testing across browsers, form validation testing, error handling scenarios, and load testing for concurrent requests.

10.4 Results Analysis

The model demonstrates reasonable performance for a Naive Bayes classifier on this complex regression problem converted to classification. The R-squared score of 0.29 indicates moderate predictive power, which is acceptable given the synthetic nature of the data and the complexity of predicting auction prices based solely on performance statistics. The confidence scores provide valuable additional information for decision-making.

11. DEPLOYMENT GUIDE (RENDER)

11.1 Prerequisites

- GitHub account
- Render account (free tier available at render.com)
- Git installed on local machine
- Project ready with all dependencies

11.2 Step 1: Prepare Project for Deployment

Create necessary configuration files:

1. Create build.sh file in project root:

```
#!/usr/bin/env
bash set -o
errexit

pip install --upgrade pip
pip install -r requirements.txt

cd frontend
npm install
npm run
build cd ..
```

2. Update requirements.txt to include:

```
flask==3.0.0
flask-
cors==4.0.0
pandas==2.1.3
numpy==1.26.2
scikit-learn==1.3.2
gunicorn==21.2.0
werkzeug==3.0.1
```

3. Verify app.py has proper PORT configuration:

```
port = int(os.environ.get('PORT', 5000))
app.run(host='0.0.0.0', port=port)
```

11.3 Step 2: Upload to GitHub

1. Initialize Git repository:

```
cd ipl-auction-
predictor git init
```

2. Create .gitignore file:

```
# Python
```

```
git add .
git commit -m 'Initial commit'
git remote add origin
https://github.com/username/repo.git git branch
-M main
git push -u origin main
```

```
d/
```

3. Commit and push to GitHub:

11.4 Step 3: Deploy on Render

1. Sign up/Login to Render:
   - Go to https://render.com
   - Sign up with GitHub account
   - Authorize Render to access repositories
2. Create New Web Service:
   - Click 'New +' button in dashboard
   - Select 'Web Service'
   - Connect your GitHub repository
   - Select 'ipl-auction-predictor' repository
3. Configure Web Service:

Setting	Value
Name	ipl-auction-predictor
Environment	Python 3
Branch	main
Build Command	chmod +x build.sh && ./build.sh
Start Command	gunicorn app:app
Instance Type	Free

4. Deploy:

- Click 'Create Web Service' button
- Monitor build logs in real-time
- Wait for 'Build successful' message
- Application will be live at https://your-app.onrender.com

11.5 Updating the Application

To update your deployed application:

```
git add .
git commit -m 'Update:
description' git push origin
main
```

11.6 Troubleshooting

- Build fails: Check build logs for missing dependencies
- Frontend not loading: Verify build.sh builds frontend correctly
- API errors: Check environment variables and PORT configuration
- Model not loading: Ensure model_artifacts/ folder is in repository
- CORS errors: Verify Flask-CORS configuration in app.py

12. FUTURE ENHANCEMENTS

12.1 Model Improvements

- Implement ensemble methods (Random Forest, Gradient Boosting)
- Deep learning models for improved accuracy
- Feature importance analysis and selection
- Hyperparameter tuning and optimization
- Cross-validation for robust performance metrics
- Integration of real IPL historical data

12.2 Application Features

- User authentication and authorization
- Save and compare multiple predictions
- Player comparison feature
- Historical auction analysis
- Team budget constraint simulation
- Export predictions to PDF/Excel
- Real-time data updates
- Interactive data visualizations and charts

12.3 Technical Enhancements

- Database integration (PostgreSQL/MongoDB)
- Caching layer for improved performance
- GraphQL API for flexible data queries
- Microservices architecture
- CI/CD pipeline implementation
- Comprehensive test suite
- API rate limiting and security
- Mobile application development

13. CONCLUSION

13.1 Project Summary

The IPL Auction Price Predictor successfully demonstrates the application of machine learning techniques to sports analytics. The project encompasses the complete software development lifecycle, from data generation and model training to frontend development and deployment. The system provides a functional and user-friendly platform for predicting player auction prices based on comprehensive performance metrics.

13.2 Key Achievements

- Developed a complete end-to-end machine learning application
- Implemented Gaussian Naive Bayes classification for price prediction
- Created an intuitive and responsive web interface
- Built a robust RESTful API with comprehensive error handling
- Achieved production-ready deployment architecture
- Processed 50,000 player records with 32 features
- Implemented real-time predictions with confidence scoring

13.3 Learning Outcomes

This project provided valuable experience in multiple areas including machine learning model development and evaluation, full-stack web application development, RESTful API design and implementation, data preprocessing and feature engineering, cloud deployment and DevOps practices, and user interface design and user experience optimization.

13.4 Final Remarks

The IPL Auction Price Predictor serves as a strong foundation for further development in sports analytics. With the proposed enhancements and integration of real-world data, this system could evolve into a professional-grade tool for IPL teams and cricket analysts. The modular architecture ensures easy maintenance and scalability for future improvements.

14. REFERENCES & APPENDICES

14.1 Technologies & Libraries

- scikit-learn: Machine Learning in Python - https://scikit-learn.org/
- Flask: Web development framework - https://flask.palletsprojects.com/
- React: JavaScript library for UIs - https://react.dev/
- NumPy: Numerical computing - https://numpy.org/
- pandas: Data analysis library - https://pandas.pydata.org/
- Framer Motion: Animation library - https://www.framer.com/motion/
- Gunicorn: Python WSGI HTTP Server - https://gunicorn.org/
- Render: Cloud hosting platform - https://render.com/

14.2 Project Repository Structure

Complete project directory structure:

```
ipl-auction-predictor/
|
|-- app.py (Flask backend server)
|-- model.py (ML model implementation)
|-- players_dataset.csv (50,000 player dataset)
|-- requirements.txt (Python dependencies)
|-- build.sh (Render build script)
|-- README.md (Project documentation)
|-- .gitignore (Git ignore file)
|
|-- model_artifacts/ (Trained model files)
| |-- naive_bayes_model.pkl
| |-- scaler.pkl
| |-- label_encoders.pkl
| |-- price_bins.pkl
| |-- feature_columns.pkl
|
|-- frontend/ (React application)
|-- package.json
|-- .env.example
|
|-- public/
| |-- index.html
|
|-- src/
|-- App.js
|-- App.css
|-- index.js
|-- index.css
```

14.3 Appendix A: Sample Dataset Record

Attribute	Value
player_id	PLR000123
age	25
role	All-Rounder
country	India
domestic_matches	100
batting_average	47.37
batting_strike_rate	135.5
wickets_taken	85
economy_rate	7.2
auction_price_lakhs	450.75

Acknowledgement

We express our sincere gratitude to **Euphoria Genx**, Haldia Institute of Technology, faculty mentors, and internship supervisors for their guidance, support, and encouragement throughout the project.

Declaration

We hereby declare that this project report titled **“Performance-Based Analysis of IPL Players and Auction Pricing”** is an original work carried out by us under the guidance of our mentors and has not been submitted elsewhere for any academic award.

— END OF DOCUMENTATION —